

1. **Fetch from a component:** fetch: a request in two waits · fetch does not fail on 404 or 500: check `res.ok` · Where the call goes: an effect or an event handler · Loading, error, empty, success · A reusable data hook: `{ data, loading, error, reload }` · fetch vs Axios: what a library adds
2. **One API client:** One API client module · Environment variables in the front end (and why they are public) · The dev proxy: `/api` → `localhost:3000` without CORS · Logging in from the front end · Sending the token: Authorization: Bearer · Handling 401: log out and go to the log-in page
3. **Sending data:** POST, PUT, PATCH and DELETE from a form · After a write: refetch, or update local state · Optimistic updates (and rolling back) · Server validation errors on the right field
4. **Races, search and pages:** Race conditions: the stale response · Cancelling with `AbortController` · Debounced search · Pagination controls: page, limit, total

Fetch from a component

fetch: a request in two waits

`fetch(url, options)` **sends an HTTP request from the browser and returns a promise of a Response; reading the body (`await res.json()`) is a second promise.**

- `const res = await fetch(url) → res.status, res.ok, res.headers.get(name).`
- `const data = await res.json() → the parsed body.` Forgetting this `await` gives you a pending promise, not data.
- A 204 No Content has no body: calling `res.json()` on it throws.
- Watch the request in the **Network** tab of DevTools (see [DevTools: the Network panel](#)).

```
import { useState } from 'react';

function App() {
  const [tasks, setTasks] = useState([]);
  const [info, setInfo] = useState('');

  async function load() {
    const res = await fetch('/api/tasks');           // wait 1: status and headers
    console.log('status', res.status, 'ok', res.ok);
    console.log('X-Total-Count', res.headers.get('X-Total-Count'));
    const data = await res.json();                  // wait 2: the body, parsed as JSON
    setInfo(res.status + ' ' + res.statusText + ': ' + data.length + ' tasks');
    setTasks(data);                                // new state: React re-renders
  }

  return (
    <main>
      <button onClick={load}>Load tasks</button>
      <p>{info}</p>
      <ul>{tasks.map((t) => <li key={t.id}>{t.title}</li>)}</ul>
    </main>
  );
}
```

Example. In the Try it box, press Load tasks: the console shows the status and the X-Total-Count header (available after the first `await`), then the list appears once the body is parsed. Every box in this section talks to a small mock API of tasks that answers `/api/...` requests (it starts afresh when you edit the code).

Common mistake. Writing `const tasks = fetch('/api/tasks').json()`. `fetch` returns a promise, which has no `.json()` method (TypeError), and even `(await fetch(url)).json()` is still a promise: you need both awaits, `const res = await fetch(url); const tasks = await res.json();`.

fetch does not fail on 404 or 500: check `res.ok`

The promise of `fetch` **rejects only when no response arrives** (network down, DNS failure, CORS block,

cancelled). A 404 or a 500 is a response, so it **resolves**: you check `res.ok` (status 200–299) and throw yourself.

What fetch does in each case

What happened	Promise	<code>res.ok</code>	Your catch runs?
200 with the task	resolves	true	no (nothing to catch)
404 Not Found / 401 / 500	resolves	false	only if you throw
Network down, server not running	rejects (TypeError: Failed to fetch)	(no response)	yes
Blocked by CORS	rejects (TypeError)	(no response for your code)	yes
Cancelled with AbortController	rejects (AbortError)	(no response)	yes

```
import { useState } from 'react';

async function getTask(id) {
  const res = await fetch('/api/tasks/' + id);
  if (!res.ok) {
    // 404 and 500 RESOLVE: turn them into errors ourselves, with the server's message
    const body = await res.json().catch(() => ({}));
    throw new Error(body.error || 'HTTP ' + res.status);
  }
  return res.json();
}

function App() {
  const [id, setId] = useState('2');
  const [result, setResult] = useState('');

  async function show() {
    try {
      const task = await getTask(id);
      setResult('Found: ' + task.title);
    } catch (err) {
      setResult('Error: ' + err.message);
    }
  }

  return (
    <main>
      <label>Task id <input value={id} onChange={(e) => setId(e.target.value)} /></label>
      <button onClick={show}>Fetch</button>
      <p>{result}</p>
    </main>
  );
}
```

Example. In the Try it box, fetch task 2 (found) and task 99 (the helper turns the 404 into an error with the server's message). Then delete the whole `if (!res.ok) { ... }` block and fetch 99 again: the page says "Found: undefined", because the error body `{ error: ... }` was treated as a task.

Common mistake. Wrapping a fetch in `try/catch` and believing errors are handled. Without a `res.ok` check, a 401 or a 500 sails past the `catch`, the error body is stored as if it were data, and the UI breaks somewhere else ("tasks.map is not a function", because `{ error }` is not an array).

Where the call goes: an effect or an event handler

Fetch in an **effect** when the data is needed because something is **on screen** (the page opened, a filter changed); fetch in an **event handler** when the request is the result of something the user **did** (submit, delete, mark as done).

```
import { useEffect, useState } from 'react';

function App() {
  const [tasks, setTasks] = useState([]);
  const [showDone, setShowDone] = useState(false);
```

```
// Needed because it is on screen: an effect that runs again when showDone changes.
useEffect(() => {
  let ignore = false;
  fetch('/api/tasks?done=' + showDone)
    .then((res) => res.json())
    .then((data) => { if (!ignore) setTasks(data); });
  return () => { ignore = true; };
}, [showDone]);

// Caused by a click: the request goes in the event handler.
async function toggle(task) {
  const res = await fetch('/api/tasks/' + task.id, {
    method: 'PATCH',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ done: !task.done }),
  });
  if (res.ok) setTasks(tasks.filter((t) => t.id !== task.id)); // it moved to the other list
}

return (
  <main>
    <label><input type="checkbox" checked={showDone} onChange={(e) => setShowDone(e.target.checked)} /> Show done
    tasks</label>
    <ul>{tasks.map((t) => (
      <li key={t.id}>{t.title} <button onClick={() => toggle(t)}>{t.done ? 'Undo' : 'Done'}</button></li>
    ))}</ul>
  </main>
);
}
```

Example. In the Try it box, the list is fetched in an effect that depends on `showDone`: tick the checkbox and the effect runs again with `?done=true`. Marking a task as done is a user action, so the `PATCH` is sent from the button's `onClick`, which also knows which task to change.

Common mistake. Leaving a dependency out of the array (`useEffect(load, [])` when `load` reads `showDone`). The effect then runs once with the first value and never again, so the filter "does nothing". List every value from the component that the effect reads.

Loading, error, empty, success

A component that loads data is always in one of **four states**: **loading** (no answer yet), **error** (it failed), **empty** (it worked, and there is nothing), **success** (there is data). Render each one on purpose.

Four states, four renders

State	When	Show
loading	request sent, no answer yet	"Loading tasks..." or a skeleton; disable buttons that would repeat it
error	rejected promise, or <code>res.ok === false</code>	what failed, in words, and a Try again button
empty	success with <code>[]</code>	why it is empty and what to do next
success	success with items	the data

```
import { useEffect, useState } from 'react';

const SEARCH = ''; // try 'zzz' for the empty state

function App() {
  const [state, setState] = useState({ status: 'loading', tasks: [], error: null });
  const [attempt, setAttempt] = useState(0);

  useEffect(() => {
    let ignore = false;
    setState({ status: 'loading', tasks: [], error: null });
    fetch('/api/tasks?search=' + SEARCH)
      .then((res) => {
        if (!res.ok) throw new Error('HTTP ' + res.status);
        return res.json();
      });
  });
}
```

```

    })
    .then((tasks) => { if (!ignore) setState({ status: 'success', tasks, error: null }); })
    .catch((error) => { if (!ignore) setState({ status: 'error', tasks: [], error }); });
    return () => { ignore = true; };
  }, [attempt]);

  if (state.status === 'loading') return <p aria-busy="true">Loading tasks...</p>;
  if (state.status === 'error') return (
    <p role="alert">Could not load the tasks ({state.error.message}).{' '}
      <button onClick={() => setAttempt(attempt + 1)}>Try again</button></p>
  );
  if (state.tasks.length === 0) return <p>No tasks match. Create your first one!</p>;
  return <ul>{state.tasks.map((t) => <li key={t.id}>{t.title}</li>)}</ul>;
}

```

Example. The box waits 1.2 s per request, so you see the loading state. Change SEARCH to 'zzz' for the empty state. For the error state, open the Fetch lab, which has switches for a network failure and a 500.

Common mistake. Showing "No tasks" while the request is still running. If the initial state is `tasks = []` and you only check `tasks.length === 0`, the empty message flashes before every load and also appears after an error. Decide the state first (loading? error?), and only then look at the length.

A reusable data hook: { data, loading, error, reload }

A custom hook is a function whose name starts with `use` and that calls other hooks; wrapping the loading/error/data logic in one (`useApi(path)`) lets every component load data in one line.

```

import { useEffect, useState } from 'react';

// The loading / error / data logic, written once.
function useApi(path) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [version, setVersion] = useState(0);

  useEffect(() => {
    const controller = new AbortController();
    setLoading(true);
    setError(null);
    fetch(path, { signal: controller.signal })
      .then((res) => { if (!res.ok) throw new Error('HTTP ' + res.status); return res.json(); })
      .then((json) => { setData(json); setLoading(false); })
      .catch((err) => { if (err.name !== 'AbortError') { setError(err); setLoading(false); } });
    return () => controller.abort();
  }, [path, version]);

  return { data, loading, error, reload: () => setVersion((v) => v + 1) };
}

function TaskCount() {
  const { data, loading } = useApi('/api/tasks?done=false');
  return <p>{loading || !data ? '...' : data.length} tasks to do</p>;
}

function App() {
  const { data, loading, error, reload } = useApi('/api/tasks');
  return (
    <main>
      <TaskCount />
      <button onClick={reload} disabled={loading}>Reload</button>
      {error && <p role="alert">{error.message}</p>}
      {data && <ul>{data.map((t) => <li key={t.id}>{t.title}</li>)}</ul>}
    </main>
  );
}

```

Example. The box defines `useApi` once and uses it twice: `TaskCount` shows how many tasks are not done, `App` shows the whole list with a `Reload` button. Each call has its own loading state and its own request (see the console).

Common mistake. Calling the hook conditionally (`if (id) { const t = useApi('/api/tasks/' + id); }`) or naming it without `use`. Hooks must be called in the same order on every render, at the top level of a component or another hook; to skip the request, pass a value the hook understands (for example `null`) and let the effect return early.

fetch vs Axios: what a library adds

`fetch` is the built-in web standard. **Axios** is a popular library around HTTP requests that adds conveniences: **JSON by default, rejecting on non-2xx, interceptors, a base URL and timeouts.** Both send the same HTTP.

The same needs, two tools

Need	fetch	Axios
Install	built in	<code>npm install axios</code>
Send JSON	<code>body: JSON.stringify(x) + Content-Type header</code>	<code>axios.post(url, x)</code>
Read JSON	<code>await res.json()</code>	<code>res.data</code> (already parsed)
404 / 500	resolves; check <code>res.ok</code>	rejects; <code>err.response.status</code>
No connection	rejects (<code>TypeError</code>)	rejects (<code>err.response</code> is undefined)
Base URL, default headers	your own wrapper	<code>axios.create({ baseURL, headers })</code>
Token on every request, 401 handling	your own wrapper	request and response interceptors
Cancel	<code>AbortController</code> + signal	the same signal option

```
// The same client with Axios (npm install axios). Not runnable here: the boxes have no npm.
import axios from 'axios';

export const http = axios.create({
  baseURL: import.meta.env.VITE_API_URL ?? '/api', // every path is relative to this
});

// Request interceptor: runs before every request.
http.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = 'Bearer ' + token;
  return config;
});

// Response interceptor: runs after every response; non-2xx arrive in the error branch.
http.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) window.dispatchEvent(new Event('auth:logout'));
    return Promise.reject(error);
  },
);

// Usage: JSON both ways, and a 404 throws.
const { data: tasks } = await http.get('/tasks', { params: { done: false } });
const { data: created } = await http.post('/tasks', { title: 'Learn Axios' });
```

Example. `await axios.get('/api/tasks/99')` rejects with `err.response.status === 404` and `err.response.data = { error: 'Task 99 not found' }`. The `fetch` version resolves; you write `if (!res.ok) throw ... yourself`. Same request on the wire, different promise behaviour.

Common mistake. Mixing the two habits: checking `res.ok` on an Axios response (it has no `ok`; it already threw), or reading `res.data` from a `fetch` response (undefined: you must `await res.json()`). Know which one a file uses, and keep it to one per project.

One API client

One API client module

An API client is one module (for example `src/api/client.js`) that every request goes through: it knows the base URL, sets the JSON headers, adds the token, checks `res.ok` and turns failures into one error shape. Components call `api.listTasks()`, never fetch directly.

```
// src/api/client.js
const BASE_URL = import.meta.env.VITE_API_URL ?? '/api';

export class ApiError extends Error {
  constructor(status, message, errors = []) {
    super(message);
    this.status = status;    // 0 = no response at all
    this.errors = errors;    // [{ field, message }] from a 400
  }
}

export async function request(path, { method = 'GET', body, token, signal } = {}) {
  const headers = { Accept: 'application/json' };
  if (body !== undefined) headers['Content-Type'] = 'application/json';
  if (token) headers.Authorization = 'Bearer ' + token;
  let res;
  try {
    res = await fetch(BASE_URL + path, { method, headers, signal, body: body === undefined ? undefined :
JSON.stringify(body) });
  } catch (err) {
    if (err.name === 'AbortError') throw err;          // cancelled on purpose
    throw new ApiError(0, 'Cannot reach the server. Check your connection.');
```

```
  }
  const data = res.status === 204 ? null : await res.json().catch(() => null);
  if (!res.ok) throw new ApiError(res.status, (data && data.error) || 'HTTP ' + res.status, (data && data.errors)
|| []);
  return data;
}

// src/api/tasks.js: the API in words
export const listTasks = (query = '', opts) => request('/tasks' + query, opts);
export const createTask = (task, opts) => request('/tasks', { method: 'POST', body: task, ...opts });
export const deleteTask = (id, opts) => request('/tasks/' + id, { method: 'DELETE', ...opts });
```

```
import { useEffect, useState } from 'react';

// ---- api.js: the only code that knows about HTTP ----
class ApiError extends Error {
  constructor(status, body) {
    super(body.error || 'HTTP ' + status);
    this.status = status;
    this.errors = body.errors || [];
  }
}

async function request(path, { method = 'GET', body } = {}) {
  const res = await fetch('/api' + path, {
    method,
    headers: body ? { 'Content-Type': 'application/json' } : {},
    body: body ? JSON.stringify(body) : undefined,
  });
  const data = res.status === 204 ? null : await res.json().catch(() => ({}));
  if (!res.ok) throw new ApiError(res.status, data || {});
  return data;
}

const api = {
  listTasks: () => request('/tasks'),
  createTask: (title) => request('/tasks', { method: 'POST', body: { title } }),
};

// ---- a component: only WHAT it wants ----
```

```
function App() {
  const [tasks, setTasks] = useState([]);
  const [message, setMessage] = useState('');
  useEffect(() => { api.listTasks().then(setTasks); }, []);

  async function add(title) {
    try {
      const task = await api.createTask(title);
      setTasks([...tasks, task]);
      setMessage('Created task ' + task.id);
    } catch (err) {
      setMessage(err.status + ': ' + err.message + ' ' + err.errors.map((e) => e.message).join(', '));
    }
  }
  return (
    <main>
      <button onClick={() => add('Learn fetch')}>Add "Learn fetch"</button>
      <button onClick={() => add('')}>Add an empty task</button>
      <p>{message}</p>
      <ul>{tasks.map((t) => <li key={t.id}>{t.title}</li>)}</ul>
    </main>
  );
}
```

Example. The box puts a small client and the component in one file (a real project splits them). "Add an empty task" gets a 400; the component shows the status, the message and the field error without knowing anything about HTTP.

Common mistake. Building the client but still calling `fetch` "just this once" in a page. That one call misses the token, the 401 handling and the error shape, and it is exactly the request that fails in production. Make it a rule: components import from `api/`, never call `fetch`.

Environment variables in the front end (and why they are public)

A front-end build tool such as Vite puts variables whose names start with `VITE_` into `import.meta.env` (for example `import.meta.env.VITE_API_URL`) and writes their values into the JavaScript bundle: anyone who opens the site can read them.

- Only `VITE_*` names reach `import.meta.env`; the value is fixed at **build** time.
- Commit a `.env.example` with fake values; keep real `.env.local` files out of Git.
- In Create React App the prefix was `REACT_APP_` and the object `process.env`: same idea, same rule.
- More on secrets on the server: [Secrets and configuration](#).

```
# .env.development (used by npm run dev)
VITE_API_URL=/api

# .env.production (used by npm run build)
VITE_API_URL=https://api.example.com/api

# NEVER in a front-end .env: these would be readable by every visitor
# VITE_JWT_SECRET=... VITE_DB_PASSWORD=...

// src/api/client.js: read it once, with a fallback
const BASE_URL = import.meta.env.VITE_API_URL ?? '/api';
if (import.meta.env.DEV) console.log('API at', BASE_URL);
```

Example. After `npm run build`, search the generated `dist/assets/index-....js` for `api.example.com`: the URL is there as plain text. That is fine for a URL; it would be a leak for a key.

Common mistake. Putting an API key or the JWT secret in `VITE_SECRET_KEY` and believing `.env` keeps it private. `.env` is private on your disk, but the build copies every `VITE_` value into public JavaScript. If the browser needs to prove something, it must get a short-lived token from your server, never a long-lived secret.

The dev proxy: /api → localhost:3000 without CORS

In development, the front end (Vite on localhost:5173) and the API (localhost:3000) are different origins. A dev proxy makes the dev server forward /api/... requests to the API, so the browser only ever talks to one origin and CORS never comes into play.

Two ways to reach the API in development

	Direct: fetch('http://localhost:3000/api/tasks')	Proxy: fetch('/api/tasks')
Origin the browser talks to	localhost:3000 (cross-origin)	localhost:5173 (same origin)
CORS needed?	yes: Access-Control-Allow-Origin, preflight for JSON + token	no
Address in the code	hard-coded or from an env variable	relative; the proxy decides
Network tab shows	requests to port 3000, plus OPTIONS preflights	requests to port 5173

```
// vite.config.js
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      // /api/tasks → http://localhost:3000/api/tasks
      '/api': 'http://localhost:3000',

      // If the API's routes have no /api prefix (/tasks), strip it:
      // '/api': { target: 'http://localhost:3000', changeOrigin: true,
      //           rewrite: (path) => path.replace(/^\/api/, '') },
    },
  },
});
```

Example. Without a proxy, fetch('http://localhost:3000/api/tasks') from the page on port 5173 fails with "blocked by CORS policy" until the API adds cors(). With the proxy above and fetch('/api/tasks'), the same API answers and the console stays clean, with no change on the server.

Common mistake. Restarting the API, adding Access-Control-Allow-Origin: * everywhere, or installing a "CORS unblock" browser extension to fix a development CORS error. The proxy is the clean fix in development; in production, configure CORS for the real origin. Also remember the proxy only applies to **relative** URLs: fetch('http://localhost:3000/...') bypasses it.

Logging in from the front end

Logging in is a POST of the credentials ({ email, password }) to the API; a successful answer carries a token (and usually the user), which the front end keeps and sends with later requests.

```
import { useState } from 'react';

function App() {
  const [email, setEmail] = useState('ana@example.com');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const [session, setSession] = useState(null); // { token, user }
  const [sending, setSending] = useState(false);

  async function login(e) {
    e.preventDefault();
    setError('');
    setSending(true);
    try {
      const res = await fetch('/api/auth/login', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ email, password }),
      });
    }
  }
}
```

```

    const data = await res.json();
    if (!res.ok) throw new Error(data.error);
    setSession(data);          // a real app also keeps data.token (context, storage)
  } catch (err) {
    setError(err.message);
  } finally {
    setSending(false);
  }
}

if (session) return <p>Welcome, {session.user.name} ({session.user.role}). Token: <code>{session.token}</code>
</p>;
return (
  <form onSubmit={logIn}>
    <label>Email <input value={email} onChange={(e) => setEmail(e.target.value)} /></label>
    <label>Password <input type="password" value={password} onChange={(e) => setPassword(e.target.value)} />
  </label>
    <button disabled={sending}>{sending ? 'Logging in...' : 'Log in'}</button>
    {error && <p role="alert">{error}</p>}
  </form>
);
}

```

Example. The box talks to the mock API's POST /api/auth/login. Log in as ana@example.com with password password123 (an admin) or as leo@example.com (a student). Try a wrong password (401) and an empty one (400 with field errors).

Common mistake. Telling the user "No account with that email" on one failure and "Wrong password" on another. That lets anyone find out which emails are registered. Show one message for both, as the API does: "Invalid email or password".

Sending the token: Authorization: Bearer

After logging in, the front end proves who it is on each request by sending the token in the Authorization header: Authorization: Bearer <token> (the word **Bearer**, one space, the token).

```

import { useState } from 'react';

function App() {
  const [token, setToken] = useState('admin-token');
  const [result, setResult] = useState('');

  async function createUser() {
    const headers = { 'Content-Type': 'application/json' };
    if (token) headers.Authorization = 'Bearer ' + token; // the word Bearer, a space, the token
    const res = await fetch('/api/users', {
      method: 'POST',
      headers,
      body: JSON.stringify({ name: 'Iris', email: 'iris' + Date.now() + '@example.com', password: 'longpassword'
    })),
  });
  const data = await res.json();
  setResult(res.status + ': ' + (res.ok ? 'created user ' + data.id : data.error));
}

return (
  <main>
    <label>Token{' '}
      <select value={token} onChange={(e) => setToken(e.target.value)}>
        <option value="admin-token">admin-token (Ana, admin)</option>
        <option value="student-token">student-token (Leo, student)</option>
        <option value="">no token</option>
        <option value="expired-token">expired-token</option>
      </select>
    </label>
    <button onClick={createUser}>POST /api/users</button>
    <p>{result}</p>
  </main>
);

```

```

    </main>
  );
}

```

Example. The box creates a user with `POST /api/users`, which only admins may do. Send it with Ana's admin token (**201**), with Leo's student token (**403**: known, not allowed), with no token and with an expired one (**401**: unknown). Read [401 vs 403](#) for what each means.

Common mistake. Writing `Authorization: token` or `Authorization: Bearer: token`, or building it as `'Bearer' + token` without the space. The server splits on the space and finds no token, so every request gets 401 although the user "is logged in". Log the exact header once in the Network tab.

Handling 401: log out and go to the log-in page

A 401 on a request that **carried a token** means the token is no longer accepted (expired, revoked, the secret changed). The client should handle it in **one place**: forget the token, update the UI to "logged out", and send the user to the log-in page with a short explanation.

```

import { useState } from 'react';

// The client knows HOW to spot an expired token; the app decides what logging out means.
function createClient({ getToken, onUnauthorized }) {
  return async function request(path, options = {}) {
    const token = getToken();
    const headers = { 'Content-Type': 'application/json' };
    if (token) headers.Authorization = 'Bearer ' + token;
    const res = await fetch('/api' + path, { ...options, headers });
    if (res.status === 401 && token) { // we sent a token and it was refused
      onUnauthorized();
      throw new Error('Session expired');
    }
    const data = await res.json();
    if (!res.ok) throw new Error(data.error || 'HTTP ' + res.status);
    return data;
  };
}

function App() {
  const [token, setToken] = useState('expired-token'); // saved days ago
  const [notice, setNotice] = useState('');
  const request = createClient({
    getToken: () => token,
    onUnauthorized: () => { setToken(null); setNotice('Your session has expired. Please log in again.'); },
  });

  async function addUser() {
    try {
      await request('/users', { method: 'POST', body: JSON.stringify({ name: 'Iris', email: 'iris@example.com', password: 'longpassword' }) });
      setNotice('User created. ');
    } catch (err) { console.log('request failed:', err.message); }
  }

  if (!token) return <p role="status">{notice} <button onClick={() => { setToken('admin-token'); setNotice(''); }}>Log in again</button></p>;
  return <p><button onClick={addUser}>Create a user</button> {notice}</p>;
}

```

Example. The box starts with a token "saved days ago" that the server no longer accepts. Press Create a user: the client sees 401, calls `onUnauthorized`, and the app shows the logged-out state with an explanation. Log in again (admin token) and the request works (201; a second click gives 409 because the email now exists).

Common mistake. Redirecting to /login on **every** 401, including the one from the log-in form. A user who mistypes the password is then "redirected" to the page they are on, and the message about the wrong password never appears. Check that a token was sent (or skip the handler for the log-in call).

Sending data

POST, PUT, PATCH and DELETE from a form

A write is a fetch with a method, a Content-Type: application/json header and a body made with JSON.stringify; the form's onSubmit handler calls e.preventDefault() and sends it.

```
import { useEffect, useState } from 'react';

function App() {
  const [tasks, setTasks] = useState([]);
  const [title, setTitle] = useState('');

  useEffect(() => {
    fetch('/api/tasks').then((res) => res.json()).then(setTasks);
  }, []);

  async function add(e) {
    e.preventDefault(); // no page reload
    const res = await fetch('/api/tasks', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' }, // "the body is JSON"
      body: JSON.stringify({ title }), // object → JSON text
    });
    if (!res.ok) return console.log('not created:', res.status);
    const created = await res.json(); // 201: the task, with its id
    setTasks([...tasks, created]);
    setTitle('');
  }

  async function remove(id) {
    const res = await fetch('/api/tasks/' + id, { method: 'DELETE' });
    if (res.status === 204) setTasks(tasks.filter((t) => t.id !== id)); // 204: no body
  }

  return (
    <form onSubmit={add}>
      <input value={title} onChange={(e) => setTitle(e.target.value)} aria-label="New task" />
      <button>Add</button>
      <ul>{tasks.map((t) => (
        <li key={t.id}>{t.title} <button type="button" onClick={() => remove(t.id)}>Delete</button></li>
      ))}</ul>
    </form>
  );
}
```

Example. In the Try it box, type a title and press Enter or Add: a POST (201) adds it with its new id. Delete sends DELETE /api/tasks/:id and gets 204, so the code removes the item from the list without parsing any body. The Delete buttons have type="button": inside a form, a plain <button> is a submit button.

Common mistake. Sending body: { title } (an object) instead of body: JSON.stringify({ title }). fetch converts the object to the string "[object Object]", and the server answers 400 for malformed JSON. A body is always text.

After a write: refetch, or update local state

When a write succeeds, the screen must show the change. Either **refetch the data from the server (simple, always correct, one more request) or **update local state** from the response (fast, but you must reproduce the**

server's rules: sorting, filters, counts).

```
import { useEffect, useState } from 'react';

function App() {
  const [tasks, setTasks] = useState([]);
  const [strategy, setStrategy] = useState('local');

  const load = () => fetch('/api/tasks?sort=createdAt&order=desc').then((r) => r.json()).then(setTasks);
  useEffect(() => { load(); }, []);

  async function add() {
    const res = await fetch('/api/tasks', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ title: 'Added at ' + new Date().toLocaleTimeString() }),
    });
    if (!res.ok) return;
    if (strategy === 'refetch') {
      await load(); // a second request: the server's truth
    } else {
      const created = await res.json(); // use the 201 body: no second request
      setTasks((prev) => [created, ...prev]); // newest first, as the server sorts
    }
  }

  return (
    <main>
      <label><input type="radio" checked={strategy === 'local'} onChange={() => setStrategy('local')} /> Update local state</label>
      <label><input type="radio" checked={strategy === 'refetch'} onChange={() => setStrategy('refetch')} /> Refetch the list</label>
      <p><button onClick={add}>Add a task</button> (watch the [network] lines below)</p>
      <ol>{tasks.map((t) => <li key={t.id}>{t.title}</li>)}</ol>
    </main>
  );
}
```

Example. In the Try it box, switch between the two strategies and add a few tasks. "Update local state" sends one request (POST) and puts the new task first, as the server sorts; "Refetch the list" sends a POST and then a GET. Both show the same list; the console shows the cost.

Common mistake. Building the new item on the client (`setTasks([...tasks, { title }])`) instead of using the server's response. It has no id (so no key and nothing to delete later), no createdAt, and it shows even when the server rejected the write. Wait for success and use what the server returned.

Optimistic updates (and rolling back)

An optimistic update changes the screen before the server answers, assuming success, and rolls back to the previous state (with a message) if the request fails.

```
import { useEffect, useState } from 'react';

function App() {
  const [tasks, setTasks] = useState([]);
  const [error, setError] = useState('');

  useEffect(() => {
    fetch('/api/tasks').then((r) => r.json())
      // A task someone else deleted a minute ago: our list still shows it.
      .then((data) => setTasks([...data, { id: 99, title: 'Deleted elsewhere', done: false }]));
  }, []);

  async function toggle(task) {
    const before = tasks; // 1. remember
    setTasks(tasks.map((t) => (t.id === task.id ? { ...t, done: !t.done } : t))); // 2. show it now
    setError('');
  }
}
```

```

try {
  const res = await fetch('/api/tasks/' + task.id, {
    method: 'PATCH',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ done: !task.done }),
  });
  if (!res.ok) throw new Error('HTTP ' + res.status);
} catch (err) {
  setTasks(before); // 3. roll back, and say so
  setError('Could not update "' + task.title + '" (' + err.message + ').');
}
}

return (
  <main>
    {error && <p role="alert">{error}</p>}
    <ul>{tasks.map((t) => (
      <li key={t.id}><label><input type="checkbox" checked={t.done} onChange={() => toggle(t)} /> {t.title}
    </label></li>
    ))}</ul>
  </main>
);
}

```

Example. The box loads the tasks and adds one more that "someone else deleted a minute ago" (id 99). Tick any real task: the tick appears at once, and the PATCH succeeds a second later. Tick "Deleted elsewhere": the tick appears, the server answers 404, and the code rolls back with a message.

Common mistake. Rolling back to a snapshot taken before **several** overlapping requests. Tick task 1 and task 2 quickly; if the first fails, `setTasks(before)` also undoes task 2, which succeeded. For overlapping actions, roll back only the item that failed (`setTasks((prev) => prev.map(...))` for that id).

Server validation errors on the right field

When the API rejects a form with 400 { error, errors: [{ field, message }] }, turn the list into an object keyed by field ({ title: 'title must not be empty' }) and show each message next to its input, marked with `aria-invalid` and linked with `aria-describedby`.

```

import { useState } from 'react';

function App() {
  const [form, setForm] = useState({ title: '', userId: '42' });
  const [errors, setErrors] = useState({});
  const [saved, setSaved] = useState(null);
  const change = (e) => setForm({ ...form, [e.target.name]: e.target.value });

  async function submit(e) {
    e.preventDefault();
    setErrors({});
    const res = await fetch('/api/tasks', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ title: form.title, userId: Number(form.userId) }),
    });
    const data = await res.json();
    if (res.status === 400) {
      const byField = {};
      for (const { field, message } of data.errors) byField[field || 'form'] = message;
      return setErrors(byField); // { title: '...', userId: '...' }
    }
    if (!res.ok) return setErrors({ form: data.error });
    setSaved(data);
  }

  const props = (name) => ({ id: name, name, onChange: change, value: form[name],
    'aria-invalid': errors[name] ? true : undefined, 'aria-describedby': errors[name] ? name + '-error' : undefined
  });
}

```

```
const message = (name) => errors[name] && <span id={name + '-error'} className="error"> {errors[name]}</span>;

return (
  <form onSubmit={submit} noValidate>
    <p><label htmlFor="title">Title</label> <input {...props('title')} />{message('title')}</p>
    <p><label htmlFor="userId">Owner</label> <select {...props('userId')}>
      <option value="1">Ana (1)</option><option value="2">Leo (2)</option><option value="42">User 42</option>
    </select>{message('userId')}</p>
    {errors.form && <p role="alert">{errors.form}</p>}
    <button>Create</button>
    {saved && <p>Created task {saved.id}: {saved.title}</p>}
  </form>
);
}
```

Example. In the Try it box, submit with an empty title and the owner "User 42": the server answers 400 with two errors, and each appears under its own field (the form has noValidate, so the browser lets the request through). Fix both and submit: 201.

Common mistake. Showing the whole error object as one alert ("Validation failed") or `JSON.stringify(data.errors)` above the form. The user then has to work out which field is meant. Map each field to its input; only errors without a field belong at the top.

Races, search and pages

Race conditions: the stale response

A **race condition** happens when two requests are in flight and the answers arrive in a different order than the requests left: the **older** (stale) response lands last and overwrites the newer one, so the screen shows results for something the user no longer asked for.

```
import { useEffect, useState } from 'react';

// Broad searches are slower on a real server; here we simulate it: "s" waits 1.5 s more than "status".
const slowFor = (q) => Math.max(0, 6 - q.length) * 300;
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

function App() {
  const [query, setQuery] = useState('');
  const [tasks, setTasks] = useState([]);
  const [shownFor, setShownFor] = useState('');

  useEffect(() => {
    let ignore = false; // the fix, part 1
    async function search() {
      const res = await fetch('/api/tasks?search=' + encodeURIComponent(query));
      const data = await res.json();
      await wait(slowFor(query));
      if (ignore) return; // the fix, part 2: delete this line to see the bug
      setTasks(data);
      setShownFor(query);
    }
    search();
    return () => { ignore = true; }; // the query changed: this answer is now stale
  }, [query]);

  return (
    <main>
      <label>Type "status" quickly: <input type="search" value={query} onChange={(e) => setQuery(e.target.value)} /></label>
      <p>Box: "{query}" · list shows results for: "{shownFor}"</p>
      <ul>{tasks.map((t) => <li key={t.id}>{t.title}</li>)}</ul>
    </main>
  );
}
```

Example. The box simulates a server where short searches are slower (it adds a delay of 300 ms per missing letter). Type "status" quickly: the list ends up right. Now delete the line if (ignore) return; and type again: the list jumps around and can end with the results for "s" while the box says "status". The Fetch lab below draws each request on a timeline.

Common mistake. Believing that requests sent in order come back in order, because it works on your machine where the API answers in 5 ms. On a real network with real data the order is not guaranteed, and the bug shows up only for users. Protect every effect that fetches by a changing value.

Cancelling with AbortController

An `AbortController` **cancels a fetch: pass `controller.signal` in the options, call `controller.abort()` (in the effect's cleanup), and the request's promise rejects with an error named `AbortError`, which you ignore.**

```
import { useEffect, useState } from 'react';

function App() {
  const [query, setQuery] = useState('');
  const [tasks, setTasks] = useState([]);
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController();
    setError(null);
    fetch('/api/tasks?search=' + encodeURIComponent(query), { signal: controller.signal })
      .then((res) => {
        if (!res.ok) throw new Error('HTTP ' + res.status);
        return res.json();
      })
      .then(setTasks)
      .catch((err) => {
        if (err.name === 'AbortError') return; // we cancelled it on purpose: not an error
        setError(err);
      });
    return () => controller.abort(); // the query changed, or the component left
  }, [query]);

  return (
    <main>
      <label>Search <input type="search" value={query} onChange={(e) => setQuery(e.target.value)} /></label>
      {error && <p role="alert">{error.message}</p>}
      <ul>{tasks.map((t) => <li key={t.id}>{t.title}</li>)}</ul>
    </main>
  );
}
```

Example. In the Try it box (each answer takes 0.8 s), type a word quickly and read the console: every keystroke cancels the request before it ("→ cancelled"), and only the last one gets a 200.

Common mistake. Creating one `AbortController` outside the effect (or in a `useRef`) and reusing it. Once aborted, a controller stays aborted: every later request using its signal is cancelled immediately. Create a new controller in each effect run.

Debounced search

Debouncing waits until the input has stopped changing for a short time (say 300–500 ms) before acting, so a search box sends **one** request when the user pauses instead of one per keystroke.

```
import { useEffect, useState } from 'react';

// Gives back `value`, but only once it has stopped changing for `ms`.
function useDebounced(value, ms) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const timer = setTimeout(() => setDebounced(value), ms);
  }, [value]);
  return debounced;
}
```

```

    return () => clearTimeout(timer);      // a new keystroke restarts the wait
  }, [value, ms]);
  return debounced;
}

function App() {
  const [query, setQuery] = useState('');
  const search = useDebounce(query, 400);
  const [tasks, setTasks] = useState([]);
  const [requests, setRequests] = useState(0);

  useEffect(() => {
    const controller = new AbortController();
    setRequests((n) => n + 1);
    fetch('/api/tasks?search=' + encodeURIComponent(search), { signal: controller.signal })
      .then((res) => res.json())
      .then(setTasks)
      .catch((err) => { if (err.name !== 'AbortError') console.error(err); });
    return () => controller.abort();
  }, [search]);

  return (
    <main>
      <label>Search <input type="search" value={query} onChange={(e) => setQuery(e.target.value)} /></label>
      <p>Requests sent: {requests}{query !== search && ' ' · waiting for you to stop typing...}</p>
      <ul>{tasks.map((t) => <li key={t.id}>{t.title}</li>)}</ul>
    </main>
  );
}

```

Example. In the Try it box, type "status" quickly: the counter goes up by just one, after you stop (the first request is the empty search when the Try it box loads). Type slowly, one letter per second, and every letter becomes a request. Change 400 to 0 and every keystroke sends one again.

Common mistake. Writing `const search = debounce(fetchTasks, 400)` in the body of the component. The body runs on every render, so each keystroke creates a **new** debounced function with its own timer, and nothing is debounced at all. Keep the timer in an effect (as above) or in a `useRef`.

Pagination controls: page, limit, total

A paginated list keeps the current **page** in state, asks the API for `?page=&limit=`, reads the **total** count the API reports, and computes the number of pages (`Math.ceil(total / limit)`) to show "Page 2 of 3" and to **disable** Previous on the first page and Next on the last.

```

import { useEffect, useState } from 'react';

const LIMIT = 2;

function App() {
  const [page, setPage] = useState(1);
  const [result, setResult] = useState({ items: [], total: 0 });
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    let ignore = false;
    setLoading(true);
    fetch('/api/tasks?page=' + page + '&limit=' + LIMIT)
      .then(async (res) => {
        const total = Number(res.headers.get('X-Total-Count')); // this API sends the total in a header
        const items = await res.json();
        if (!ignore) { setResult({ items, total }); setLoading(false); }
      });
    return () => { ignore = true; };
  }, [page]);

  const pages = Math.max(1, Math.ceil(result.total / LIMIT));
  const from = result.total === 0 ? 0 : (page - 1) * LIMIT + 1;

```

```
const to = Math.min(page * LIMIT, result.total);

return (
  <main aria-busy={loading}>
    <ul>{result.items.map((t) => <li key={t.id}>{t.title}</li>)}</ul>
    <p>Showing {from}-{to} of {result.total}</p>
    <nav aria-label="Pagination">
      <button onClick={() => setPage(page - 1)} disabled={page <= 1 || loading}>← Previous</button>
      { ' ' }Page {page} of {pages}{ ' ' }
      <button onClick={() => setPage(page + 1)} disabled={page >= pages || loading}>Next →</button>
    </nav>
  </main>
);
}
```

Example. The box asks for two tasks per page. Five tasks make three pages: on page 1 Previous is disabled; on page 3 ("Showing 5–5 of 5") Next is disabled. While a page loads (0.6 s here) both are disabled.

Common mistake. Deciding that "Next" is possible when the page came back full (`items.length === limit`). On the last page that happens to be full (4 items, 2 per page), Next stays enabled and leads to an empty page. Use the total: `page < Math.ceil(total / limit)`.